# Cross-Model Cross-Stream Learning for Self-Supervised Human Action Recognition

Mengyuan Liu, Hong Liu, *Member, IEEE*, and Tianyu Guo

***Abstract*—Considering the instance-level discriminative ability, contrastive learning methods, including MoCo and SimCLR, have been adapted from the original image representation learning task to solve the self-supervised skeleton-based action recognition task. These methods usually use multiple data streams (i.e., joint, motion, and bone) for ensemble learning, meanwhile, how to construct a discriminative feature space within a single stream and effectively aggregate the information from multiple streams remains an open problem. To this end, this article first applies a new contrastive learning method called bootstrap your own latent (BYOL) to learn from skeleton data, and then formulate SkeletonBYOL as a simple yet effective baseline for self-supervised skeleton-based action recognition. Inspired by SkeletonBYOL, this article further presents a cross-model and cross-stream (CMCS) framework. This framework combines cross-model adversarial learning (CMAL) and cross-stream collaborative learning (CSCL). Specifically, CMAL learns single-stream representation by cross-model adversarial loss to obtain more discriminative features. To aggregate and interact with multistream information, CSCL is designed by generating similarity pseudolabel of ensemble learning as supervision and guiding feature generation for individual streams. Extensive experiments on three datasets verify the complementary properties between CMAL and CSCL and also verify that the proposed method can achieve better results than state-of-the-art methods using various evaluation protocols.**

***Index Terms*—Multistream, self-supervised learning, skeleton-based action recognition.**

## I. Introduction

HUMAN action recognition has wide applications in human-computer interaction, video surveillance, and health care [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]. Different from RGB videos or depth data, 3-D skeleton data encodes high-level representations of human actions and it is robust to background clutter, illumination changes, and appearance variation [8], [11], [12]. In addition, with the popularity of depth sensors [13] and the development of advanced human pose estimation algorithms [14], [15], [16], [17], [18], the acquisition of skeleton data has become accessible. Thus, skeleton-based action recognition has gradually attracted more and more attention.

Previous works have explored various approaches for skeleton-based action recognition. Traditional methods [19], [20], [21] focus on handcrafted features to model skeleton sequence patterns. With the advancements in deep learning, recurrent neural networks (RNNs) and long-short term memory (LSTM) have been used to model temporal dynamics [22], [23], [24]. Further, convolutional neural networks (CNNs)-based methods [25], [26], [27], [28] convert skeleton sequences to RGB images and use CNNs to learn spatio-temporal features. In recent years, due to the similarity between the human skeleton and graph structure, some methods [29], [30], [31] capture spatiotemporal relationships leveraging the graph structure of human skeletons based on graph convolutional networks (GCNs), which have achieved superior performances. Nevertheless, the above methods are all trained in a supervised manner, which is costly to obtain for large-scale datasets. To address this issue, self-supervised skeleton-based action recognition has gained attention [32], [33], [34], [35], [36], [37], [38], [39], [40], as it can better utilize unlabeled data to learn more generalized feature representations.

In the domain of self-supervised skeleton-based action recognition, certain approaches [32], [33] employ generative pretext tasks, such as reconstruction and prediction, to learn representations from unlabeled skeleton sequences. On the other hand, as contrastive self-supervised learning gains popularity, other works [34], [35], [36] adopt the contrastive learning framework to achieve better results in this field. They apply data augmentations on the skeleton sequence to construct contrastive pairs (i.e., positive pairs and negative pairs) and hope to learn general representations by pulling the positive pairs closer and pushing the negative pairs away. There are also some methods [37], [38], [39], [40] that combine the generative task with the contrastive task to extract discriminative representations in the multitask learning manner.

Among them, compared with the generative methods, the contrastive methods pay more attention to the instance-level information rather than the detailed information and then construct a more discriminative feature space that is more suitable for downstream tasks, so it has received extensive attention. Presently, the contrastive methods utilized in self-supervised skeleton-based action recognition [34], [35], [36] require special attention when handling negative samples. This is achieved through techniques like using large batch sizes, memory banks,

Received 2 January 2024; revised 25 July 2024; accepted 20 September 2024. Date of publication 17 October 2024; date of current version 20 November 2024. This work was supported in part by the Natural Science Foundation of Guangdong Province under Grant 2024A1515012089 and in part by the National Natural Science Foundation of China under Grant 62203476. This article was recommended by Associate Editor V. Fuccella. *(Corresponding authors: Hong Liu; Tianyu Guo.)*

The authors are with the State Key Laboratory of General Artificial Intelligence, Peking University, Shenzhen Graduate School, Shenzhen 518055, China (e-mail: hongliu@pku.edu.cn; levigty@stu.pku.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/THMS.2024.3467334.

Digital Object Identifier 10.1109/THMS.2024.3467334

2168-2291 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

or negative sampling strategies to effectively retrieve and utilize negative samples during training. These approaches help enhance the learning process and improve the quality of learned representations for the task. Different from previous methods, bootstrap your own latent (BYOL) [41] does not use negative samples, and learns better feature representation through a simple framework, which provides a new idea for the development of contrastive learning in the field of skeleton action recognition. Therefore, this article adopts BYOL as the baseline to build our method. Further, this article rethinks existing contrastive self-supervised skeleton-based action recognition methods and finds several unsolved problems.

1) How to construct a discriminative feature space within a single stream? Although the above methods have improved the skeleton representation ability to a certain extent, it is believed that the ability of self-supervised methods has not been fully explored. Using the BYOL framework [41] directly is not necessarily the most suitable for skeleton action recognition tasks, and the learned feature discrimination could be further improved. Constructing a more discriminative feature space within a single stream is a very challenging problem, especially for the contrastive learning framework without negative samples.

2) How to interact and aggregate the information from multiple streams? In skeleton action recognition, joint, motion, and bone data are three common types of data used to describe human motion. Joint data provides information about the position of each joint in the human body. Motion data describes how the joints are moving over time, while bone data describe the spatial relationship between adjacent joints. They are complementary because they each provide unique information and by combining these three types of data, this article can get a more complete understanding of the action. However, existing methods often use simple ensemble learning, which cannot perform information interaction well. If such complementary information could be fully utilized and explored, a better feature representation would be obtained. Thus, how to better fuse information between streams needs to be carefully considered.

To this end, this article presents a cross-model cross-stream (CMCS) framework to enhance both single-stream and multistream representations for self-supervised skeleton-based action recognition. Specifically, SkeletonBYOL is first proposed, following BYOL [41] as the baseline. Second, cross-model adversarial learning (CMAL) is proposed to effectively learn single-stream representation as shown in Fig. 1(a). Unlike BYOL, CMAL is proposed to make the feature "Attract" or "Repel," and then get a more discriminative feature space. Third, cross-stream collaborative learning (CSCL) is proposed to aggregate cross-stream information as shown in Fig. 1(b). The feature distribution of the three streams (i.e., joint, motion, and bone) is fused into an ensemble similarity pseudolabel and the pseudolabel is used as a supervision signal to optimize the feature space of every single stream. Then the pseudolabel continuously updates and iterates to enhance the representations.

In summary, our main contributions are twofold.

1) This article presents a CMCS framework to effectively enhance both single-stream and multistream representations for self-supervised skeleton-based action recognition.

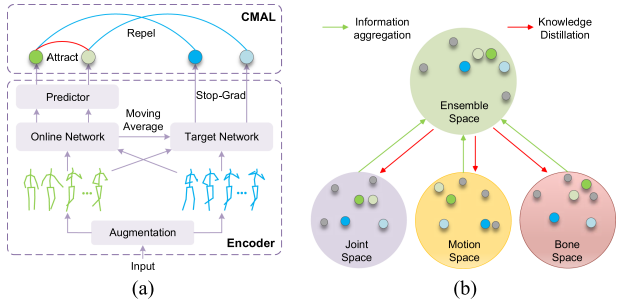



Fig. 1. Illustration of our CMCS framework, which mainly involves an encoder block, a CMAL block, and a CSCL block. (a) For a single stream, better intrastream representation is learned by CMAL with "Attract" and "Repel" operations on features extracted from an encoder. To facilitate observation, two skeleton sequences are colored in green and blue. The green dot means the feature extracted from the green skeleton sequence and the blue dot means the feature extracted from the blue skeleton sequence. (b) For multiple streams, CSAL aggregates interstream information and makes the feature space of a single stream to be consistent with the ensemble space. (Best viewed in color).

2) This article formulates a simple yet effective baseline, i.e., SkeletonBYOL. Based on the encoder of SkeletonBYOL, our CMCS framework further involves CMAL and CSCL to learn distinctive representations from both single-stream and multistreams.

## II. Related Work

### A. Supervised Methods

Traditional methods for skeleton-based action recognition [19], [20] rely on handcrafted features. With the advancement of deep learning, some approaches [22], [23] use RNNs to model the temporal features of skeleton sequences. However, RNNs suffer from gradient vanishing issues. To address this, other methods [25], [26], [42] convert the 3-D skeleton sequence into pseudoimages and utilize CNNs, achieving competitive results. Nonetheless, both RNNs and CNNs do not fully capture the inherent graph structure of skeleton data. Recently, GCNs, particularly ST-GCN [29], have gained popularity in skeleton-based action recognition, leading to the emergence of various GCN-based methods [30], [31], [43]. This article uses ST-GCN as the encoder for feature extraction to demonstrate the effectiveness of our proposed framework.

### B. Self-Supervised Methods

Self-supervised learning falls under the category of unsupervised learning and involves defining pretext tasks as learning objectives to acquire meaningful representations from unlabeled data. For the skeleton sequence data, there are generative pretext tasks [32], [33], [44], [45], [46], contrastive pretext tasks [34], [35], [36], [47], [48], [49], [50], [51], and multiple pretext tasks. For generative pretext tasks, LongTGAN [32] employs an encoder–decoder architecture to reconstruct the input sequence and derive meaningful feature representations. P&C [33] also relies on a reconstruction pretext task but adopts a training strategy to weaken the decoder, encouraging the encoder to learn more discriminative features. Cheng et al. [44] proposed

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

a pretext task involving predicting the motion of 3-D skeletons, utilizing the hierarchical transformer for encoding the skeleton sequences. Yang et al. [45] introduced a novel technique called skeleton cloud colorization to facilitate the learning of skeleton representations. Xu et al. [46] proposed the motion capsule autoencoder (MCAE) to address the "transformation invariance" challenge in the motion representations. Indeed, methods based on frame-level generation, like those mentioned previously, may not be inherently suitable for representation learning in the context of skeleton-based action recognition [47]. There might be limitations in effectively capturing the underlying spatiotemporal relationships and structures present in the skeleton data using such frame-level generation approaches.

For contrastive pretext tasks, AS-CAL [34] proposes to use plenty of spatiotemporal augmentations and use momentum LSTM and memory bank. SkeletonCLR [35] is built based on MoCo v2 [52], [53] and uses *Shear* and *Crop* as the augmentations, which achieves competitive performance. Further, CrosSCLR [35] introduces a cross-view consistent knowledge mining strategy to enhance the performance of SkeletonCLR. ST-CL [47] leverages the spatiotemporal continuity of motion tendency as the supervisory signal to acquire action-specific features. Thoker et al. [36] proposed interskeleton contrastive learning, which learns from multiple different input skeleton representations in a cross-contrastive manner. Xu et al. [48] proposed the spatiotemporal decouple-and-squeeze contrastive learning (SDS-CL) framework, which jointly contrasts spatial-squeezing features, temporal-squeezing features, and global features while incorporating a new attention mechanism to improve feature learning. Moliner et al. [49] proposed to use vanilla BYOL and asymmetric augmentation, but its gain mainly comes from using a larger number of encoder channels. CMD [50] also uses the MoCo v2 framework and proposes cross-modal mutual distillation to improve the performance. Pang et al. [51] proposed a novel contrastive GCN-Transformer network (ConGT), focusing on designing a stronger encoder.

The generative pretext tasks pay more attention to the detailed information of the skeleton sequence, and the contrastive pretext task pays more attention to the discriminative information of the skeleton sequence at the instance level. In order to learn more general features, there are also several methods to do multiple pretext tasks. MS$^2$L [37] proposes to use multiple pretext tasks (i.e., motion prediction, jigsaw puzzle recognition, and contrastive learning) and combine them to encourage the Bi-GRU encoder to capture more suitable features. CP-STN [38] integrates contrastive learning paradigms and generative pretext tasks within a single framework by employing asymmetric spatial and temporal augmentations, allowing the network to extract discriminative representations. Tanfous et al. [39] proposed a taxonomy of self-supervised learning for action recognition, reconciling AS-CAL and P&C. PCRP [40] introduces a novel framework that employs prototypical contrast and reverse prediction to fully learn inherent semantic similarity in the context of self-supervised action recognition.

Our method belongs to the category of using contrastive pretext tasks. Different from the existing methods [34], [35], [47], our method focuses on exploring a new baseline to obtain more discriminative features within a single stream and focuses on how to interact and aggregate information between different streams to obtain a more discriminative feature space than ensemble learning. The proposed method has a simpler structure and is applicable to a variety of encoders.

## III. Method

### A. SkeletonBYOL

Contrastive learning has gained widespread popularity due to its ability to perform instance discrimination effectively. Motivated by this, we introduce SkeletonBYOL, a method to learn single-stream 3-D action representations. Our approach is inspired by a simple yet effective technique called BYOL [41]. SkeletonBYOL is a nonnegative samples method for skeleton representation, where each sample is considered as its own positive sample, and we leverage different augmentations of the same sample during training. In each training step, the positive samples are embedded close to each other. SkeletonBYOL consists of four major components.

*Augmentation:* Data augmentations $\mathcal{T}$ are used to randomly transform the given skeleton sequence $s$ into different augments $x$ and $x'$ that are considered as positive pairs. For skeleton data, *Shear* and *Crop* are adopted as the augmentation strategy following SkeletonCLR [35].

1) *Shear:* The shear augmentation is a linear transformation applied to the spatial dimensions of 3-D coordinates of body joints. It slants the shape of the joints with a random angle. The transformation matrix for shear is defined as

$$A = \begin{bmatrix} 1 & a_{12} & a_{13} \\ a_{21} & 1 & a_{23} \\ a_{31} & a_{32} & 1 \end{bmatrix}. \tag{1}$$

The shear factors, denoted as $a_{12}, a_{13}, a_{21}, a_{23}, a_{31}$, and $a_{32}$, are randomly sampled from a uniform distribution within the range $[-\beta, \beta]$, where $\beta$ is the shear amplitude. The transformation matrix $A$ is then formed with these shear factors and applied to the skeleton sequence along the channel dimension.

2) *Crop:* For the crop augmentation, it is commonly used in image classification tasks as it enhances diversity while preserving the distinguishing features of the original samples. In the case of temporal skeleton sequences, the first several frames are flipped and concatenated to the original sequence. Also, the last several frames are flipped and further concatenated to the above sequence. Finally, the sequence is cropped back to the original length. Let $T$ denote the length of the skeleton sequence. The length of padding is defined as $T/\gamma$, where $\gamma$ is the padding ratio, and it is set to 6.

*Online network:* The online network is defined by a set of weights $\theta$ and is comprised of three components: an encoder $f_\theta$, a projector $g_\theta$, and a predictor $q_\theta$. The encoder $f_\theta$ embeds $x$ into feature space: $y_\theta = f_\theta(x)$. The projector $g_\theta$ is a multilayer perceptron (MLP), which consists of a linear layer followed by batch normalization, a rectified linear unit (ReLU), and a final linear layer. $g_\theta$ is used to project the feature vector to a new feature space: $z_\theta = g_\theta(y_\theta)$. The predictor $q_\theta$ is used to predict the output of the target network, which has the same structure as $g_\theta$. The parameters $\theta$ of the online network are updated via gradients.

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1:** Pseudocode of the Encoder and CMAL.

**Input:** $\mathcal{D}$ is a training set
**Output:** $\mathcal{L}_{\text{total}}$ is the loss value
$\mathcal{L}_{\text{total}} \leftarrow 0$
**for** a minibatch $s$ in $\mathcal{D}$ with $n$ samples **do**
  get the augmented sample $x$ and $x'$
  use the online network to obtain the online feature $z_\theta$ and $z'_\theta \leftarrow x, x'$
  use the online predictor to obtain the predicted feature $p_\theta$ and $p'_\theta \leftarrow z_\theta, z'_\theta$
  use the target network to obtain the online feature $z_\xi$ and $z'_\xi \leftarrow x, x'$
  use the mean square error to obtain the loss value in a mini-batch:
  $\mathcal{L}_{\text{CMAL}} = \text{mse}(p_\theta, p'_\theta) - \frac{1}{2}(\text{mse}(p_\theta, z_\xi) + \text{mse}(p'_\theta, z'_\xi))$
  loss backward
  update the online network with gradient
  update the target network with momentum update
  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \mathcal{L}_{\text{CMAL}}$
**end for**
**return** $\mathcal{L}_{\text{total}}$

*Target network:* The target network is comprised of two components: an encoder $f_\xi$ and a projector $g_\xi$. They have the same architecture as that in an online network but use a different set of weights $\xi$. The target network provides the regression targets to train the online network: $y'_\xi = f_\xi(x')$, $z'_\xi = g_\xi(y'_\xi)$. Its parameters $\xi$ are an exponential moving average of the online parameters $\theta$. More precisely, given a target decay rate $\tau \in [0, 1]$, after each training step we perform the update

$$\xi \leftarrow \tau\xi + (1 - \tau)\theta. \tag{2}$$

*BYOL:* Finally, we define the following mean squared error between the $l_2$-normalized prediction $q_\theta(z_\theta)$ and target projections $z'_\xi$:

$$\begin{aligned}\mathcal{L}_{\theta,\xi} &= \|q_\theta(z_\theta) - sg(z'_\xi)\|_2^2 \\ &= 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), sg(z'_\xi)\rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|sg(z'_\xi)\|_2}\end{aligned} \tag{3}$$

where $\langle , \rangle$ denotes the inner product and $sg(\cdot)$ means stop-gradient. We symmetrize the loss $\mathcal{L}_{\theta,\xi}$ in (3) by feeding $x'$ to the online network and $x$ to the target network to compute $\tilde{\mathcal{L}}_{\theta,\xi}$. At each training step, a stochastic optimization step is performed to minimize $\mathcal{L}_{\text{BYOL}} = \mathcal{L}_{\theta,\xi} + \tilde{\mathcal{L}}_{\theta,\xi}$.

## B. CMAL

SkeletonBYOL predicts the output of the target network from an online network. It discards the negative samples completely and achieves good performance. However, it is difficult to understand intuitively why this direct prediction is valid. It is difficult to obtain a more discriminative feature space. Inspired by RAFT [54], we hope to better use the framework and optimize two goals at the same time: 1) keeping representations of two augmented samples of the same action sequence aligned and 2) keeping a sample's online representation differing from its target representation. Thus, CMAL is proposed to learn single-stream representation more effectively.

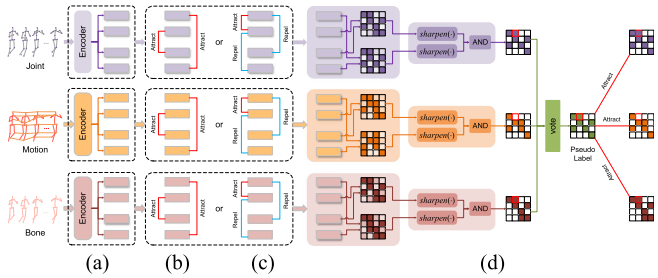



Fig. 2. Comparison between our proposed CMCS framework and our proposed simple baseline, i.e., SkeletonBYOL. Our CMCS framework consists of (a) encoder, (c) CMAL, and (d) CSCL. Meanwhile, the SkeletonBYOL consists of (a) encoder and (b) BYOL. Noting that the encoder contains augmentation, online network, and target network. The red line means "Attract" and the blue line means "Repel." (Best viewed in color). (a) Encoder. (b) BYOL. (c) CMAL (Ours). (d) CSCL (Ours).

For an input sequence $s$, features $z_\theta$, $z'_\theta$, $z_\xi$, and $z'_\xi$ are extracted. Then a prediction head is used to obtain $q_\theta(z_\theta)$ and $q_\theta(z'_\theta)$. In order to obtain a more discriminative feature space and avoid collapse solutions, we first hope to align representations of $x$ and $x'$ similar by $\mathcal{L}^1_{\text{min}}$

$$\mathcal{L}^1_{\text{min}} = \|q_\theta(z_\theta) - q_\theta(z'_\theta)\|_2^2. \tag{4}$$

On the other hand, we also hope the feature of the online encoder could be different from the target encoder by $\mathcal{L}^2_{\text{min}}$ to obtain a more discriminative feature space

$$\begin{aligned}\mathcal{L}^2_{\text{min}} = \frac{1}{2}\Big(&\|q_\theta(z_\theta) - sg(z_\xi)\|_2^2 \\ &+ \|q_\theta(z'_\theta) - sg(z'_\xi)\|_2^2\Big).\end{aligned} \tag{5}$$

To jointly optimize these two goals, our final cross-model adversarial loss is as follows: $\mathcal{L}_{\text{CMAL}} = \alpha\mathcal{L}^1_{\text{min}} + \beta\mathcal{L}^2_{\text{min}}$, where $\alpha$ and $\beta$ are the coefficients to balance the loss. The two models are updated through momentum, but at the same time, the extracted features are expected to maximize the similarity as much as possible, which forms an implicit confrontation, effectively avoiding model collapse and constructing a more discriminative feature space. The pseudocode for using both the encoder and the CMAL is in Algorithm 1.

## C. CSCL

Considering easily obtained multistream skeleton data (i.e., joint, motion, and bone), fusing complementary information preserved in different streams can definitely learn better spatiotemporal information. We design CSCL to mine more ideal feature space in multiple streams.

*1) Intrastream Similarity:* In single-stream, we gather $q_\theta(z_\theta)$, $q_\theta(z'_\theta)$, $sg(z_\xi)$, $sg(z'_\xi)$ in a batch and do L2-normalization to get $Q, Q', K, K' \in \mathbb{R}^{N\times C}$, respectively. $N$ represents the batch size and $C$ represents the feature dimension. As shown in Fig. 3(a), we first calculate the cosine similarity

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

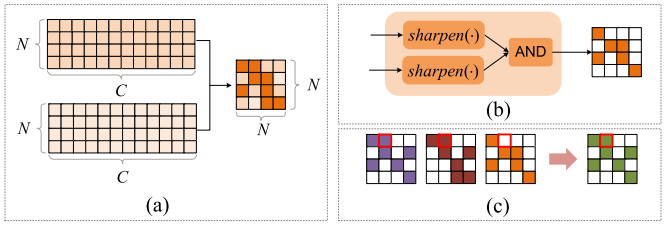



Fig. 3. Three key components of our proposed CSCL. (Best viewed in color). (a) Intra-stream similarity. (b) Intra-stream fusion. (c) Inter-stream voting.

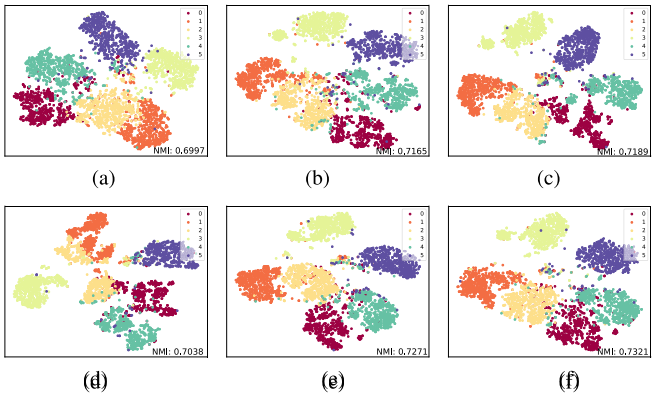



Fig. 4. t-SNE visualization of embeddings on NTU-60 xview. Different methods extract features for six categories of samples, and the visualization results after feature dimensionality reduction are shown in the figure. Noting that 3s-CMAL means using CMAL on three streams. Compared with previous SkeletonCLR and 3s-CrosSCLR methods, our proposed SkeletonBYOL and CMAL do not show obvious improvements, while our proposed 3s-CMAL and CMCS methods show better performances by separating different colored dots more clearly from each other. (a) SkeletonCLR. (b) SkeletonBYOL. (c) CMAL. (d) 3s-CrosSCLR. (d) 3s-CMAL. (e) CMCS.

in a mini-batch for a single stream. Take "joint stream" as an example

$$\mathcal{S}_{\text{joint}} = QK'^{\top}, \mathcal{S}'_{\text{joint}} = Q'K^{\top} \tag{6}$$

where $\mathcal{S}_{\text{joint}}, \mathcal{S}'_{\text{joint}} \in \mathbb{R}^{N\times N}$. Correspondingly, the similarity matrix $\mathcal{S}_{\text{bone}}, \mathcal{S}'_{\text{bone}}$ of bone stream and $\mathcal{S}_{\text{motion}}, \mathcal{S}'_{\text{motion}}$ of motion stream can also be obtained in a similar way.

*2) Intrastream Fusion:* We hope to get a sharper similarity matrix to obtain a more discriminative feature space. As shown in Fig. 3(b), we use sharpen$(\cdot)$ to make the similarity matrix sharp in a single stream

$$\hat{\mathcal{S}}_J = \text{sharpen}(\mathcal{S}_{\text{joint}}) \text{ AND } \text{sharpen}(\mathcal{S}'_{\text{joint}}) \tag{7}$$

here, sharpen$(\cdot)$ operation means setting the diagonal elements and the top-$k$ elements of each row of the similarity matrix to 1, and the others to 0 to obtain a sharper similarity matrix. Then perform an "AND" operation to obtain the similarity pseudolabel within the stream. In a similar way, we can get $\hat{\mathcal{S}}_B$ and $\hat{\mathcal{S}}_M$, respectively.

*3) Interstream Voting:* In Fig. 3(c), we propose to fuse the similarity matrix of three streams to obtain a more ideal similarity matrix $\hat{\mathcal{S}} = \text{vote}(\hat{\mathcal{S}}_J, \hat{\mathcal{S}}_B, \hat{\mathcal{S}}_M)$, where vote$(\cdot)$ means ensembling the results to determine the value of each element is 1 or 0. Specifically, when there are more than two streams that think the element should be 1, it will be 1, otherwise, it will be 0. Undoubtedly, the result of ensemble learning is better than that of a single stream, so the sharper similarity matrix $\hat{\mathcal{S}}$ is more discriminative than $\hat{\mathcal{S}}_J$, $\hat{\mathcal{S}}_B$, and $\hat{\mathcal{S}}_M$.

*4) Optimization Goal:* A very intuitive idea is to use $\hat{\mathcal{S}}$ as a supervision signal to optimize the similarity matrix of each stream, then the optimized similarity matrix of each stream can better generate $\hat{\mathcal{S}}$. In this way, information between different streams can be effectively aggregated, thereby obtaining better feature representations. Thus, the similarity difference minimization loss can be calculated like this

$$\mathcal{L}_{\text{CSCL}} = -\frac{1}{2}\sum\left[\hat{\mathcal{S}}\log(\mathcal{S}_i) + (1-\hat{\mathcal{S}})\log(1-\mathcal{S}_i)\right] \tag{8}$$

where $\mathcal{S}_i \in \{\mathcal{S}_{\text{joint}}, \mathcal{S}'_{\text{joint}}, \mathcal{S}_{\text{motion}}, \mathcal{S}'_{\text{motion}}, \mathcal{S}_{\text{bone}}, \mathcal{S}'_{\text{bone}}\}$. This loss is expected to make the feature similarity of every single stream approach the feature similarity of the ensemble feature. Minimizing the difference in similarity rather than minimizing the difference in features is a softer way to obtain a better feature space.

*Training:* In the early training stage, the encoder of each stream is not stable and strong enough to perform cross-stream information aggregation. Thus, for the CMCS framework, which is shown in Fig. 2, the encoder is trained with the CMAL: $\mathcal{L}_1 = \mathcal{L}_{\text{CMAL}}$ in the first training stage. Then in the second training stage, the loss function is $\mathcal{L}_2 = \lambda\mathcal{L}_{\text{CMAL}} + \gamma\mathcal{L}_{\text{CSCL}}$ to start aggregating cross-stream information. Here, $\lambda$ and $\gamma$ are the coefficients to balance the loss.

## IV. Experiments

### A. Datasets

*1) PKU-MMD Dataset* [55] is composed of two subsets with nearly 20 000 action sequences across 51 action classes. Part I is considered easier for action recognition, while Part II poses more challenges due to significant view variation leading to more skeleton noise. The experiments on these subsets are conducted under the cross-subject protocol.

*2) NTU-60 Dataset* [56] includes 56 578 action sequences covering 60 action classes. It has two evaluation protocols: cross-subject (xsub) and cross-view (xview). In xsub, half of the subjects are used for training, and the remaining half are used for testing. In xview, camera 2 and 3 sequences are used for training, while camera 1 sequences are used for testing.

*3) NTU-120 Dataset* [57] consists of 120 action classes and 113 945 sequences. It also has two evaluation protocols: cross-subject (xsub) and cross-setup (xset). In xsub, actions performed by 53 subjects are used for training, while the remaining subjects are used for testing. In xset, all 32 setups are split into two halves for training and testing, respectively.

### B. Experimental Settings

*1) Pretext Training:* All experiments are conducted on the PyTorch framework [61] with a single GeForce GTX 3090 GPU. For data preprocessing, mini-batch size, and encoder, we all follow CrosSCLR [35] for fair comparisons. The representation $y_\theta$ has a feature dimension of 512. The projection head is composed of the following layers: linear layer with an output size of 2048, batch normalization layer, rectified linear unit (ReLU) activation

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON UNDER KNN EVALUATION PROTOCOL WITH JOINT STREAM

| Method | PKU-MMD(%) | | NTU-60(%) | | NTU-120(%) | |
|---|---|---|---|---|---|---|
| | part I | part II | xsub | xview | xsub | xset |
| SkeletonCLR [35] | 65.9 | 17.9 | 57.7 | 67.3 | 44.9 | 45.8 |
| **CMAL (Ours)** | **77.1** | **36.6** | **64.2** | **72.3** | **50.0** | **52.1** |

Best results are in bold.

function, final linear layer with an output dimension of 512. The architecture of the prediction head is the same as the projection head. The target decay rate $\tau$ is set to 0.996. For optimization, we use SGD with momentum (0.9), weight decay (0.0001), and a learning rate of 0.1. Since the size of the three datasets varies greatly, it is favorable to set different training epochs for them to avoid overfitting and achieve better convergence. For NTU-60, we set the training epoch to 100 (the first 60 epoch is the first training stage). For NTU-120, we set the training epoch to 80 (the first 40 epoch is the first training stage). For PKU-MMD Part I, we set the training epoch to 150 (the first 100 epoch is the first training stage). For PKU-MMD Part II, we set the training epoch to 250 (the first 200 epoch is the first training stage).

*2) Downstream Testing:* We evaluate the trained encoder under multiple protocols after the pretext training.

*a) KNN evaluation protocol:* It involves using a K-nearest neighbor (KNN) classifier on the features learned by the trained encoder. It helps assess the quality and effectiveness of the learned features in representing the data. For all reported KNN results, $K = 20$.

*b) Linear evaluation protocol:* The encoder is evaluated by training a classification head. The encoder is fixed during this process. The classification head consists of a fully connected layer followed by a softmax layer. The training of the classification head is performed using SGD with an initial learning rate of 3.0 for 100 epochs. The learning rate is reduced to 0.3 at epoch 80. The batch size is set to 128.

*c) Semisupervised evaluation protocol:* After training the encoder, a linear classifier is added to the trained encoder. The entire model is then trained using only a small percentage (1% or 10%) of randomly selected labeled data. SGD is employed for training the whole network with an initial learning rate of 0.1, which is reduced by a factor of 10 at epoch 80. The entire network is trained for 150 epochs with a batch size set to 128, except for PKU-MMD Part II under the 1% semisupervised evaluation protocol, where the batch size is set to 52 due to the limited data.

*d) Finetune evaluation protocol:* Similar to semisupervised evaluation protocol, but use all labeled data.

### C. Comparison With State-of-The-Art Methods

*1) KNN Evaluation Results:* Indeed, the KNN classifier does not involve learning additional weights, making it a simple yet effective evaluation metric for the learned features. The results presented in Table I provide fair comparisons under the specified value of $K$. It is evident that our single-stream CMAL outperforms SkeletonCLR [35] on all three datasets when using the KNN classifier. These substantial improvements achieved with a simple classifier indicate that the features learned by our CMAL model are more discriminative and capture better representations of the action data.

*2) Linear Evaluation Results:* As shown in Table II, we compare the proposed CMCS with other recent methods. For using GRU as the encoder, our CMCS outperforms other methods on the three datasets under the linear evaluation protocol. For using stronger encoders, our CMCS also leads 3s-AimCLR under many protocols. For the performance on NTU-120, our CMCS still outperforms other methods by a large margin. The results demonstrate that our CMCS method is highly competitive, especially on multiclass large-scale datasets. On the PKU-MMD dataset, Part II presents more challenges due to increased skeleton noise caused by view variation. In this scenario, our CMCS outperforms 3s-CrosSCLR, highlighting its capability to handle movement patterns arising from skeleton noise effectively. This effectiveness extends to both large-scale and small-scale datasets, indicating the generalization ability of our method. These findings collectively showcase the strong performance and versatility of our CMCS method in various action recognition scenarios.

*3) Semisupervised Evaluation Results:* Table III shows that even with a small labeled subset, our CMCS outperforms other methods. Remarkably, when using only 1% and 10% labeled data, the performance of our CMCS surpasses ISC [36], 3s-CrosSCLR [29], and 3s-Colorization [45]. These results demonstrate the effectiveness of our approach in leveraging limited labeled data and obtaining superior performance in the semisupervised learning setting. The learned spatial-temporal representations from the pretext task significantly contribute to achieving these competitive results under such challenging conditions. It also proves that our method effectively learns intra–inter stream information and obtains a more discriminative feature space.

*4) Finetuned Evaluation Results:* We have conducted a comparison of our method with supervised ST-GCN and supervised 3s-ST-GCN. These models have the same structure and parameters as our proposed CMCS method. As shown in Table IV, our CMAL achieves better results than supervised ST-GCN, which means that our framework could learn more information to get a better weight initialization. For the results of three streams, the finetuned results also outperform supervised 3s-ST-GCN and finetuned 3s-CrosSCLR, which indicates the effectiveness of our method.

*5) Transfer Learning Results:* As a common practice, transfer learning performs self-supervised pretraining on the large-scale ImageNet [62], then initializes the network with the learned weights and finally trains on the small dataset. We also conduct experiments to do pretext tasks on a larger dataset without labels, then finetune on the small dataset. In Table V, when training from scratch, the accuracy of Parts I and II is 92.3% and 53.7%. When we do the pretext task on the PKU-MMD dataset itself, we can gain the improvement of 1.8% and 4.8%. When using the NTU-60 and NTU-120 datasets for self-supervised pretraining, there is a substantial improvement in accuracy, particularly in Part II of these datasets. This significant boost in performance highlights the transferability of the learned representations.

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

TABLE II
LINEAR EVALUATION RESULTS ON THREE DATASETS

| Method | Encoder | Classifier | NTU-120(%) xsub | NTU-120(%) xset | PKU-MMD(%) part I | PKU-MMD(%) part II | Publication&Year |
|---|---|---|---|---|---|---|---|
| LongTGAN [32] | GRU | FC | – | – | 67.7† | 26.0† | AAAI'2018 |
| MS$^2$L [37] | GRU | GRU | – | – | 64.9 | 27.6 | ACMMM'2020 |
| P&C [33] | GRU | FC | 41.7 | 42.7 | – | – | CVPR'2020 |
| PCRP [40] | GRU | FC | 41.7 | 45.1 | – | – | TMM'2021 |
| AS-CAL [34] | LSTM | FC | 48.6 | 49.2 | – | – | Infomation Science'2021 |
| **CMCS (Ours)** | GRU | FC | **54.6** | **55.0** | **84.0** | **39.9** | - |
| 4s-MG-AL [58] | ST-GCN | FC | 46.2 | 49.5 | – | – | TCSVT'2022 |
| Cheng *et al.* [44] | Transformer | FC | – | – | – | – | ICME'2021 |
| MCAE [46] | MCAE | FC | 52.8 | 54.7 | – | – | NIPS'2021 |
| CP-STN [38] | ST-GCN | FC | 55.7 | 54.7 | – | – | ACML'2021 |
| ST-CL [47] | GCN | FC | 54.2 | 55.6 | – | – | TMM'2022 |
| SDS-CL [48] | DSTA | FC | 50.6 | 55.6 | – | – | TNNLS'2023 |
| 3s-CrosSCLR [35] | ST-GCN | FC | 67.9 | 66.7 | 84.9† | 21.2† | CVPR'2021 |
| Yang *et al.* [45] | DGCNN | FC | – | – | – | – | ICCV'2021 |
| Thoker *et al.* [36] | GRU+CNN | FC | 67.1 | 67.9 | 80.9 | 36.0 | ACMMM'2020 |
| 3s-AimCLR [59] | ST-GCN | FC | 68.2 | 68.8 | 87.8 | 38.5 | AAAI'2022 |
| 3s-RVTCLR+ [60] | ST-GCN | FC | 68.0 | 68.9 | – | – | CVPR'2023 |
| **CMCS (Ours)** | ST-GCN | FC | **68.5** | **71.1** | **88.1** | **53.4** | - |

† denotes the result is reproduced according to the provided code.
Best results are in bold.

TABLE III
SEMISUPERVISED EVALUATION RESULTS

| Method | PKU-MMD(%) part I | PKU-MMD(%) part II | NTU-60(%) xsub | NTU-60(%) xview |
|---|---|---|---|---|
| *1% labeled data:* | | | | |
| LongT GAN [32] | 35.8 | 12.4 | 35.2 | – |
| MS$^2$L [37] | 36.4 | 13.0 | 33.1 | – |
| ISC [36] | 37.7 | – | 35.7 | 38.1 |
| 3s-Colorization [45] | – | – | 48.3 | **52.5** |
| 3s-CrosSCLR [35] | 49.7 | 10.2 | 51.1 | 50.0 |
| **CMCS (Ours)** | **58.7** | **17.7** | **51.1** | 49.7 |
| *10% labeled data:* | | | | |
| LongT GAN [32] | 69.5 | 25.7 | 62.0 | – |
| MS$^2$L [37] | 70.3 | 26.1 | 65.2 | – |
| ISC [36] | 72.1 | – | 65.9 | 72.5 |
| 3s-Colorization [45] | – | – | 71.7 | 78.9 |
| 3s-CrosSCLR [35] | 82.9 | 28.6 | 74.4 | 77.8 |
| **CMCS (Ours)** | **86.7** | **37.2** | **76.2** | **79.6** |

Best results are in bold.

TABLE IV
FINETUNED RESULTS ON NTU-60 AND NTU-120 DATASET

| Method | NTU-60(%) xsub | NTU-60(%) xview | NTU-120(%) xsub | NTU-120(%) xset |
|---|---|---|---|---|
| ST-GCN [29] | 82.8 | 91.0 | 76.4 | 76.8 |
| **CMAL (Ours)** | **83.9** | **91.6** | **77.4** | **79.4** |
| 3s-ST-GCN [29] | 85.1 | 91.8 | 80.0 | 80.4 |
| 3s-CrosSCLR [35] | 86.2 | 92.5 | 80.5 | 80.4 |
| 3s-AimCLR [59] | 86.9 | 92.8 | 80.1 | 80.9 |
| **CMCS (Ours)** | **86.9** | **92.8** | **81.7** | **82.7** |

Best results are in bold.

TABLE V
EXPLORATION OF DIFFERENT PRETEXT DATASETS FOR FINE-TUNING ON PKU-MMD DATASET

| Pretext Task Dataset | PKU-MMD(%) part I | PKU-MMD(%) part II |
|---|---|---|
| w/o pretext task | 92.3 | 53.7 |
| PKU-MMD | 94.1 (+1.8) | 58.5 (+4.8) |
| NTU-60 xsub | 95.1 (+2.8) | 61.2 (+7.5) |
| NTU-120 xsub | 94.7 (+2.4) | 62.4 (+8.7) |

TABLE VI
ABLATION STUDY RESULTS ON NTU-60 DATASET

| Method | NTU-60(%) xsub | NTU-60(%) xview |
|---|---|---|
| SkeletonCLR [35] | 75.0 | 79.8 |
| SkeletonBYOL (Baseline: Encoder + BYOL) | 77.3 | 82.6 |
| CMAL (Encoder + CMAL) | 78.0 | 83.6 |
| CSCL (Encoder + CSCL) | 78.5 | 84.2 |
| CMCS (Encoder + CMAL + CSCL) | **78.6** | **84.5** |

Best results are in bold.

### D. Ablation Study

*1) Effectiveness of CMAL and CSCL:* We also verify the effectiveness of our proposed individual components. In Table VI, SkeletonCLR [35] achieves 75.0% on xsub and 79.8% on xview, while the proposed SkeletonBYOL achieves 77.3% on xsub and 82.6% on xview. Our CMAL outperforms these results by 0.7% and 1.0%. For the proposed CSCL, applying it to the encoder also brings significant gains, which also shows the effectiveness of the strategy. For our complete method CMCS, it achieves 78.6% and 84.5%.

*2) Comparisons With SkeletonCLR:* We compare our CMAL with the SkeletonCLR in more detail. As shown in Table VII, for the three different streams (i.e., joint, motion, and bone) of three datasets, our CMAL outperforms SkeletonCLR. For the direct ensemble results, our CMAL also outperforms 3s-SkeletonCLR. It is worth mentioning that both 3s-CrosSCLR and our CMCS explore the information interaction between streams, but our method outperforms 3s-CrosSCLR on three datasets on account that our method better aggregates information from multiple streams. More ablation on using different streams is explored in Table VIII.

*3) Choice of Feature Dimension:* Since the predictor has the same structure as the projector, there are three hyperparameters of feature dimension: the dimension of feature $y$ after encoder $d_y$, the dimension of projector's hidden layer $h_{\text{mlp}}$ and the dimension of projector's final layer $d_z$. To determine them, we

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
LINEAR EVALUATION RESULTS COMPARED WITH SKELETONCLR ON NTU-60, PKU-MMD, AND NTU-120 DATASETS

| Method | Stream | NTU-60(%) | | | | PKU-MMD(%) | | | | NTU-120(%) | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | xsub | | xview | | part I | | part II | | xsub | | xset | |
| | | acc. | Δ | acc. | Δ | acc. | Δ | acc. | Δ | acc. | Δ | acc. | Δ |
| SkeletonCLR | **J** | 68.3 | | 76.4 | | 80.9 | | 35.2 | | 56.8 | | 55.9 | |
| **CMAL (Ours)** | **J** | **73.0** | ↑ 4.7 | **79.4** | ↑ 3.0 | **86.3** | ↑ 5.4 | **47.7** | ↑ 12.5 | **61.6** | ↑ 4.8 | **63.2** | ↑ 7.3 |
| SkeletonCLR | **M** | 53.3 | | 50.8 | | 63.4 | | 13.5 | | 39.6 | | 40.2 | |
| **CMAL (Ours)** | **M** | **57.7** | ↑ 4.4 | **62.4** | ↑ 11.6 | **72.1** | ↑ 8.7 | **29.1** | ↑ 15.6 | **43.5** | ↑ 3.9 | **46.2** | ↑ 6.0 |
| SkeletonCLR | **B** | 69.4 | | 67.4 | | 72.6 | | 30.4 | | 48.4 | | 52.0 | |
| **CMAL (Ours)** | **B** | **70.0** | ↑ 0.6 | **74.0** | ↑ 6.6 | **83.3** | ↑ 10.7 | **34.1** | ↑ 3.7 | **59.2** | ↑ 10.8 | **59.2** | ↑ 7.2 |
| 3s-SkeletonCLR | **J+M+B** | 75.0 | | 79.8 | | 85.3 | | 40.4 | | 60.7 | | 62.6 | |
| **CMAL (Ours)** | **J+M+B** | **78.0** | ↑ 3.0 | **83.6** | ↑ 3.8 | **87.4** | ↑ 2.1 | **52.8** | ↑ 12.4 | **66.8** | ↑ 6.1 | **70.0** | ↑ 7.4 |
| 3s-CrosSCLR | **J+M+B** | 77.8 | | 83.4 | | 84.9 | | 21.2 | | 67.9 | | 66.7 | |
| **CMCS (Ours)** | **J+M+B** | **78.6** | ↑ 0.8 | **84.5** | ↑ 0.9 | **88.1** | ↑ 3.2 | **53.4** | ↑ 32.2 | **68.5** | ↑ 0.6 | **71.1** | ↑ 4.4 |

Δ represents the gain compared to SkeletonCLR using the same stream data. **J**, **M**, and **B** indicate joint stream, motion stream, bone stream, respectively. 3s means using three stream. Best results are in bold.

TABLE VIII
RESULTS OF DIFFERENT STREAM ON NTU-60 DATASET

| Method | Stream | xsub(%) | xview(%) |
|---|---|---|---|
| CMAL | J | 73.0 | 79.4 |
| CMAL | M | 58.8 | 62.9 |
| CMAL | B | 70.1 | 74.0 |
| CMAL ‡ | J + B | 74.8 | 80.5 |
| CMAL ‡ | J + M | 74.5 | 79.3 |
| CMAL ‡ | M + B | 73.3 | 78.1 |
| CMAL ‡ | J + M + B | 76.2 | 81.4 |
| CMCS | J + B | 77.8 | 83.1 |
| CMCS | J + M | 75.4 | 83.9 |
| CMCS | M + B | 76.4 | 81.7 |
| **CMCS** | J + M + B | **78.6** | **84.5** |

‡ represents directly ensemble the results.
Best results are in bold.

TABLE IX
FEATURE DIMENSION SEARCH EXPERIMENTS ON JOINT STREAM OF NTU-60 DATASET

| $d_y$ | $h_{mlp}$ | $d_z$ | xsub(%) | xview(%) | Avg.(%) |
|---|---|---|---|---|---|
| 256 | 1024 | 256 | 69.4 | 74.0 | 71.7 |
| 256 | 1024 | 512 | 69.4 | 74.3 | **71.9** |
| 256 | 1024 | 1024 | 69.7 | 73.7 | 71.7 |
| 512 | 1024 | 256 | 70.8 | 76.9 | 73.9 |
| 512 | 1024 | 512 | 72.7 | 75.4 | **74.1** |
| 512 | 1024 | 1024 | 71.0 | 77.0 | 74.0 |
| 512 | 512 | 512 | 71.3 | 76.7 | 74.0 |
| 512 | 2048 | 512 | **73.2** | **78.2** | **75.7** |

Best results are in bold.

TABLE X
RESULTS OF DIFFERENT TARGET DECAY RATE $\tau$ (%)

| $\tau$ | xsub | xview | Avg. |
|---|---|---|---|
| 1 | 54.7 | 58.6 | 56.7 |
| 0.999 | 70.8 | 76.1 | 73.4 |
| 0.996 | 73.0 | **79.4** | **76.2** |
| 0.99 | **73.2** | 78.2 | 75.7 |
| 0.9 | 68.9 | 74.0 | 71.5 |
| 0 | 55.2 | 54.8 | 55.0 |

Best results are in bold.

TABLE XI
RESULTS OF DIFFERENT TOP-$K$ IN CSCL ON NTU-60 DATASET (%)

| Top-$K$ | xsub | xview |
|---|---|---|
| 0 | 78.0 | 83.6 |
| 1 | **78.7** | 84.4 |
| 2 | 78.6 | **84.5** |
| 3 | 78.3 | 83.9 |
| 4 | 77.6 | 83.4 |
| 5 | 76.6 | 78.5 |

Best results are in bold.

first fix $h_{\text{mlp}}$ and find that the performance of $d_y = 512$ is better than that of $d_y = 256$ as shown in Table IX. Then we fix $d_z$ to search for the best $h_{\text{mlp}}$. We choose $d_y = 512$, $h_{\text{mlp}} = 2048$, $d_z = 512$ as our default settings.

*4) Influence of Target Decay Rate $\tau$:* The value of $\tau$ is a trade-off between updating the target network too often and updating too slowly in Table X. When $\tau = 1$, the target network remains constant and is never updated, preserving its initialization value throughout the training process. On the other hand, when $\tau = 0$, the target network is updated instantaneously to match the online network at each step, effectively copying its parameters directly. However, both extreme values, $\tau = 0$ and $\tau = 1$, lead to unstable training and poor performance. To achieve better training stability and performance, the decay rate of $\tau$ is set between 0.9 and 0.999. The value of $\tau = 0.996$ is chosen as it strikes a balance between the stability of training and effective learning, leading to improved performance in the model.

*5) Influence of Top-$K$:* From Table XI, $K = 0$ means that do not use the CSCL. It is worth mentioning that CSCL brings clear gains when $0 < K < 4$. This is because the Sharpen$(\cdot)$ and interstream voting retain high-confidence samples to learn better feature representations. When $K$ continues to increase, it has a bad impact on performance due to the introduction of too many low-confidence samples. Finally, we take $K$ to 2 according to performance.

*6) Influence of Weights in $\mathcal{L}_{CMAL}$:* We also conducted experiments to verify the performance of different weights in $\mathcal{L}_{\text{CMAL}}$. In Table XII, using only $\mathcal{L}_{\text{min}}$ ($\alpha = 1$, $\beta = 0$) or $\mathcal{L}_{\text{max}}$ ($\alpha = 0$, $\beta = 1$) does not perform well. The best performance is achieved when $\mathcal{L}_{\text{min}}$ and $\mathcal{L}_{\text{max}}$ are equally important ($\alpha = 1$, $\beta = 1$), which is also in line with our original intention when designing cross-model adversarial loss.

*7) Influence of Weights to Balance $\mathcal{L}_{CMAL}$ and $\mathcal{L}_{CSCL}$:* As shown in Table XIII, when do not use the $\mathcal{L}_{\text{CSCL}}$ ($\lambda = 1$, $\gamma = 0$), the performance is also good. However, there is a drop in

Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

TABLE XII
RESULTS OF DIFFERENT WEIGHTS TO BALANCE $\mathcal{L}_{\text{MIN}}$ AND $\mathcal{L}_{\text{MAX}}$ (%)

| $\alpha$ | $\beta$ | xsub | xview |
|---|---|---|---|
| 1 | 0 | 8.0 | 7.9 |
| 0 | 1 | 31.9 | 40.4 |
| 1 | 0.1 | 60.3 | 68.6 |
| 1 | 1 | **73.0** | **79.4** |
| 1 | 10 | 62.1 | 60.3 |

Best results are in bold.

TABLE XIII
RESULTS OF DIFFERENT WEIGHTS (%)

| $\lambda$ | $\gamma$ | xsub | xview |
|---|---|---|---|
| 1 | 0 | 78.0 | 83.5 |
| 0 | 1 | 77.4 | 82.3 |
| 1 | 0.1 | 77.2 | 81.5 |
| 1 | 1 | 64.1 | 53.6 |
| 1 | 10 | **78.6** | **84.5** |

Best results are in bold.

TABLE XIV
RESULTS OF CMAL ON NTU-60 JOINT STREAM WITH DIFFERENT BACKBONE

| Backbone | Setting | NTU-60(%) xsub | NTU-60(%) xview |
|---|---|---|---|
| ST-GCN | Supervised | 82.8 | 91.0 |
| | Finetuned | **83.9** | **91.6** |
| CTR-GCN | Supervised | 84.0 | 91.2 |
| | Finetuned | **85.5** | **91.7** |
| UNIK | Supervised | 83.2 | 89.8 |
| | Finetuned | **84.4** | **91.6** |

Best results are in bold.

performance when only using $\mathcal{L}_{\text{CSCL}}$ ($\lambda = 0$, $\gamma = 1$), but not that noticeable. Interestingly, the performance degrades when the weights of $\mathcal{L}_{\text{CMAL}}$ and $\mathcal{L}_{\text{CSCL}}$ are the same. Moreover, for our CMCS, $\lambda = 1$ and $\gamma = 10$ are the best choice. We argue that it is because information aggregation is more important in the second stage of training.

*8) Robustness to Different Backbones:* ST-GCN [29] is widely used as a backbone for skeleton-based action recognition. To verify the robustness of our method to different backbones, we explore the performance of our method when using a more advanced backbone. We tried CTR-GCN [63] (GCN-based) and UNIK [64] (CNN-based). Due to the limitation of computing resources, for fair comparisons, like ST-GCN, the input frames of CTR-GCN and UNIK are also 50 frames, and the number of channels in the network is also reduced to 1/4 of the original. Due to these changes, the fully supervised results drop somewhat compared to the results in the original paper. But thankfully, as shown in Table XIV, under the same experimental setting, our method can improve the performance of end-to-end training. This also illustrates the potential and generalizability of our method to different backbones.

*9) Qualitative Results:* In Fig. 4, we visualize the embedding distribution using t-SNE [65] with fixed settings. The t-SNE results are reported as fair comparisons, and they are obtained using the same randomly selected six class samples from the dataset. From the visual results and the calculated normalized mutual information (NMI), we can draw a conclusion that our SkeletonBYOL and CMAL's embeddings show closer clustering of the same class compared to SkeletonCLR. For multistreams, even without CA, our 3s-CMAL achieves a more competitive NMI compared to the 3s-CrosSCLR (0.7271 versus 0.7038). Our final CMCS achieves the best NMI.

## V. CONCLUSION

This article focuses on self-supervised representation learning in skeleton-based action recognition and design an CMCS framework to enhance intra–inter stream representations from unlabeled data. The SkeletonBYOL based on the BYOL framework is first constructed. On this basis, CMAL is proposed to leverage cross-model adversarial loss to learn intrastream representation more effectively. For multistreams, CSCL is proposed to aggregate interstream information to further enhance the representations. Our proposed CMCS framework outperforms state-of-the-art methods across various evaluation protocols, as demonstrated in the experiments.

## REFERENCES


[1] Z. Tu, H. Li, D. Zhang, J. Dauwels, B. Li, and J. Yuan, "Action-stage emphasized spatiotemporal VLAD for video action recognition," *IEEE Trans. Image Process.*, vol. 28, no. 6, pp. 2799–2812, Jun. 2019.

[2] M. Liu, F. Meng, and Y. Liang, "Generalized pose decoupled network for unsupervised 3D skeleton sequence-based action representation learning," *Cyborg Bionic Syst.*, vol. 2022, 2022, Art. no. 0002.

[3] J. Zhang, Y. Jia, W. Xie, and Z. Tu, "Zoom transformer for skeleton-based group activity recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 12, pp. 8646–8659, Dec. 2022.

[4] P. Hadikhani, D. T. C. Lai, and W.-H. Ong, "A novel skeleton-based human activity discovery using particle swarm optimization with Gaussian mutation," *IEEE Trans. Hum.- Mach. Syst.*, vol. 53, no. 3, pp. 538–548, Jun. 2023.

[5] S. R. Taghanaki, M. Rainbow, and A. Etemad, "Self-supervised human activity recognition with localized time-frequency contrastive representation learning," *IEEE Trans. Hum.-Mach. Syst.*, vol. 53, no. 6, pp. 1027–1037, Dec. 2023.

[6] W. Wang, J. Li, Y. Li, and X. Dong, "Predicting activities of daily living for the coming time period in smart homes," *IEEE Trans. Hum.-Mach. Syst.*, vol. 53, no. 1, pp. 228–238, Feb. 2023.

[7] M. Baruah, B. Banerjee, and A. K. Nagar, "Intent prediction in human–human interactions," *IEEE Trans. Hum.-Mach. Syst.*, vol. 53, no. 2, pp. 458–463, Apr. 2023.

[8] Z. Tu, J. Zhang, H. Li, Y. Chen, and J. Yuan, "Joint-bone fusion graph convolutional network for semi-supervised skeleton action recognition," *IEEE Trans. Multimedia*, vol. 25, pp. 1819–1831, 2022.

[9] Z. Tu et al., "Consistent 3D hand reconstruction in video via self-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 9469–9485, Aug. 2023.

[10] Q. Gao, Z. Deng, Z. Ju, and T. Zhang, "Dual-hand motion capture by using biological inspiration for bionic bimanual robot teleoperation," *Cyborg Bionic Syst.*, vol. 4, 2023, Art. no. 0052.

[11] M. Liu, F. Meng, C. Chen, and S. Wu, "Novel motion patterns matter for practical skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 1701–1709.

[12] J. Liu, X. Wang, C. Wang, Y. Gao, and M. Liu, "Temporal decoupling graph convolutional network for skeleton-based gesture recognition," *IEEE Trans. Multimedia*, vol. 26, pp. 811–823, 2023.

[13] Z. Zhang, "Microsoft kinect sensor and its effect," *IEEE Multimedia*, vol. 19, no. 2, pp. 4–10, Feb. 2012.

[14] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, Jan. 2021.

[15] H.-S. Fang, S. Xie, Y.-W. Tai, and C. Lu, "RMPE: Regional multi-person pose estimation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2334–2343.

[16] J. Martinez, R. Hossain, J. Romero, and J. J. Little, "A simple yet effective baseline for 3D human pose estimation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2640–2649.


Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.

[17] D. Pavllo, C. Feichtenhofer, D. Grangier, and M. Auli, "3D human pose estimation in video with temporal convolutions and semi-supervised training," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 7753–7762.

[18] W. Li, H. Liu, R. Ding, M. Liu, P. Wang, and W. Yang, "Exploiting temporal contexts with strided transformer for 3D human pose estimation," *IEEE Trans. Multimedia*, vol. 25, pp. 1282–1293, 2022.

[19] J. Wang, Z. Liu, Y. Wu, and J. Yuan, "Mining actionlet ensemble for action recognition with depth cameras," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 1290–1297.

[20] R. Vemulapalli, F. Arrate, and R. Chellappa, "Human action recognition by representing 3D skeletons as points in a lie group," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 588–595.

[21] R. Vemulapalli and R. Chellapa, "Rolling rotations for recognizing human actions from 3D skeletal data," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4471–4479.

[22] Y. Du, W. Wang, and L. Wang, "Hierarchical recurrent neural network for skeleton based action recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1110–1118.

[23] S. Song, C. Lan, J. Xing, W. Zeng, and J. Liu, "Spatio-temporal attention-based LSTM networks for 3D action recognition and detection," *IEEE Trans. Image Process.*, vol. 27, no. 7, pp. 3459–3471, Jul. 2018.

[24] P. Zhang, C. Lan, J. Xing, W. Zeng, J. Xue, and N. Zheng, "View adaptive neural networks for high performance skeleton-based human action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 8, pp. 1963–1978, Aug. 2019.

[25] Y. Du, Y. Fu, and L. Wang, "Skeleton based action recognition with convolutional neural network," in *Proc. Asian Conf. Pattern Recognit.*, 2015, pp. 579–583.

[26] Q. Ke, M. Bennamoun, S. An, F. Sohel, and F. Boussaid, "A new representation of skeleton sequences for 3D action recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3288–3297.

[27] C. Cao, C. Lan, Y. Zhang, W. Zeng, H. Lu, and Y. Zhang, "Skeleton-based action recognition with gated convolutional neural networks," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 29, no. 11, pp. 3247–3257, Nov. 2019.

[28] A. Banerjee, P. K. Singh, and R. Sarkar, "Fuzzy integral-based CNN classifier fusion for 3D skeleton action recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 6, pp. 2206–2216, Jun. 2021.

[29] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3482–3489.

[30] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12026–12035.

[31] Z. Chen, S. Li, B. Yang, Q. Li, and H. Liu, "Multi-scale spatial temporal graph convolutional network for skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 2, pp. 1113–1122.

[32] N. Zheng, J. Wen, R. Liu, L. Long, J. Dai, and Z. Gong, "Unsupervised representation learning with long-term dynamics for skeleton based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2644–2651.

[33] K. Su, X. Liu, and E. Shlizerman, "Predict & cluster: Unsupervised skeleton based action recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9631–9640.

[34] H. Rao, S. Xu, X. Hu, J. Cheng, and B. Hu, "Augmented skeleton based contrastive action learning with momentum LSTM for unsupervised action recognition," *Inf. Sci.*, vol. 569, pp. 90–109, 2021.

[35] L. Li, M. Wang, B. Ni, H. Wang, J. Yang, and W. Zhang, "3D human action representation learning via cross-view consistency pursuit," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 4741–4750.

[36] F. M. Thoker, H. Doughty, and C. G. Snoek, "Skeleton-contrastive 3D action representation learning," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 1655–1663.

[37] L. Lin, S. Song, W. Yang, and J. Liu, "MS$^2$L: Multi-task self-supervised learning for skeleton based action recognition," in *Proc. ACM Int. Conf. Multimedia*, 2020, pp. 2490–2498.

[38] Y. Zhan et al., "Spatial temporal enhanced contrastive and pretext learning for skeleton-based action representation," in *Proc. 13th Asian Conf. Mach. Learn.*, 2021, pp. 534–547.

[39] A. B. Tanfous, A. Zerroug, D. Linsley, and T. Serre, "How and what to learn: Taxonomizing self-supervised learning for 3D action recognition," in *Proc. IEEE Winter Conf. Appl. Computer Vis.*, 2022, pp. 2888–2897.

[40] S. Xu, H. Rao, X. Hu, J. Cheng, and B. Hu, "Prototypical contrast and reverse prediction: Unsupervised skeleton based action recognition," *IEEE Trans. Multimedia*, vol. 25, pp. 624–634, 2021.

[41] J.-B. Grill et al., "Bootstrap your own latent-a new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 21271–21284.

[42] M. Liu, H. Liu, and C. Chen, "Enhanced skeleton visualization for view invariant human action recognition," *Pattern Recognit.*, vol. 68, pp. 346–362, 2017.

[43] H. Zheng and X. Zhang, "A cross view learning approach for skeleton-based action recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 5, pp. 3061–3072, May 2022.

[44] Y.-B. Cheng, X. Chen, J. Chen, P. Wei, D. Zhang, and L. Lin, "Hierarchical transformer: Unsupervised representation learning for skeleton-based human action recognition," in Proc. *IEEE Int. Conf. Multimedia Expo.*, 2021, pp. 1–6.

[45] S. Yang, J. Liu, S. Lu, M. H. Er, and A. C. Kot, "Skeleton cloud colorization for unsupervised 3D action representation learning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 13403–13413.

[46] Z. Xu, X. Shen, Y. Wong, and M. S. Kankanhalli, "Unsupervised motion representation learning with capsule autoencoders," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 3205–3217, 2021.

[47] X. Gao, Y. Yang, Y. Zhang, M. Li, J.-G. Yu, and S. Du, "Efficient spatio-temporal contrastive learning for skeleton-based 3D action recognition," *IEEE Trans. Multimedia*, vol. 25, pp. 405–417, 2023.

[48] B. Xu, X. Shu, J. Zhang, G. Dai, and Y. Song, "Spatiotemporal decouple-and-squeeze contrastive learning for semi-supervised skeleton-based action recognition," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 11035–11048, Aug. 2024.

[49] O. Moliner, S. Huang, and K. Åström, "Bootstrapped representation learning for skeleton-based action recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshop*, 2022, pp. 4154–4164.

[50] Y. Mao, W. Zhou, Z. Lu, J. Deng, and H. Li, "CMD: Self-supervised 3D action representation learning with cross-modal mutual distillation," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 734–752.

[51] C. Pang, X. Lu, and L. Lyu, "Skeleton-based action recognition through contrasting two-stream spatial-temporal networks," *IEEE Trans. Multimedia*, vol. 25, pp. 8699–8711, 2023.

[52] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.

[53] X. Chen, H. Fan, R. Girshick, and K. He, "Improved baselines with momentum contrastive learning," 2020, *arXiv:2003.04297*.

[54] H. Shi, D. Luo, S. Tang, J. Wang, and Y. Zhuang, "Run away from your teacher: Understanding BYOL by a novel self-supervised approach," 2020, *arXiv:2011.10944*.

[55] J. Liu, S. Song, C. Liu, Y. Li, and Y. Hu, "A benchmark dataset and comparison study for multi-modal human action analytics," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 16, no. 2, pp. 1–24, 2020.

[56] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "NTU RGB d: A large scale dataset for 3D human activity analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1010–1019.

[57] J. Liu et al., "RGB D 120: A large-scale benchmark for 3D human activity understanding," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 10, pp. 2684–2701, Oct. 2020.

[58] Y. Yang, G. Liu, and X. Gao, "Motion guided attention learning for self-supervised 3D human action recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 12, pp. 8623–8634, Dec. 2022.

[59] T. Guo, H. Liu, Z. Chen, M. Liu, T. Wang, and R. Ding, "Contrastive learning from extremely augmented skeleton sequences for self-supervised action recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 1, pp. 762–770.

[60] Y. Zhu, H. Han, Z. Yu, and G. Liu, "Modeling the relative visual tempo for self-supervised skeleton-based action recognition," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 13913–13922.

[61] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 8026–8037, 2019.

[62] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[63] Y. Chen, Z. Zhang, C. Yuan, B. Li, Y. Deng, and W. Hu, "Channel-wise topology refinement graph convolution for skeleton-based action recognition," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 13359–13368.

[64] D. Yang, Y. Wang, A. Dantcheva, L. Garattoni, G. Francesca, and F. Bremond, "UNIK: A unified framework for real-world skeleton-based action recognition," 2021, *arXiv:2107.08580*.

[65] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.


Authorized licensed use limited to: Peking University. Downloaded on October 21,2026 at 06:44:37 UTC from IEEE Xplore. Restrictions apply.